SELF-HOSTED · UNOFFICIAL WHATSAPP API

WA_API SaaS

VPS Deployment Guide

A complete, step-by-step recipe to install this project on a Linux VPS that already runs 3 Emergent projects on ports 8001, 8002, 8003. This installation uses port 8004 (backend) and port 3005 (sidecar).

AT A GLANCE

Target OS	Ubuntu 22.04 / 24.04 LTS
Runtimes	Python 3.11 · Node 20 LTS · MongoDB 7
Reverse proxy	Nginx + Certbot (HTTPS)
Process manager	Supervisor
Ports (this app)	8004 (FastAPI) · 3005 (Node/Baileys)
Estimated time	30–45 minutes

Version 1.0 · February 2026 · Keep this document with your VPS credentials.

Version: 1.0 · February 2026 **Target audience:** DevOps / system admin deploying this project onto a Linux VPS **Assumptions:** Ubuntu 22.04 / 24.04 · Root or sudo access · Your VPS already runs 3 Emergent projects on ports 8001, 8002, 8003 — this new project will therefore use **port 8004 (FastAPI backend)** and **port 3005 (Node/Baileys sidecar)** to avoid conflicts.

> Naming used through this guide
>
> - App unix user : wa_api
> - App directory : /opt/wa_api
> - Backend port : 8004
> - Sidecar port : 3005
> - MongoDB DB : wa_api_db
> - Sub-domain : wa.yourdomain.com
> - Node.js version: 20.x LTS · Python: 3.11+ · Mongo: 7.x

Feel free to change any of these values — every command below references only these variables so a single find-replace adapts the entire guide.

1. Prepare the VPS

1.1 Update the system

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential git curl ca-certificates gnupg lsb-release \
                    software-properties-common ufw fail2ban unzip
```

1.2 Create a dedicated unix user (never run apps as root)

```
sudo adduser --disabled-password --gecos "" wa_api
sudo usermod -aG sudo wa_api      # optional — only if you want the user to sudo
sudo mkdir -p /opt/wa_api
sudo chown -R wa_api:wa_api /opt/wa_api
```

1.3 Configure the firewall (skip if your provider blocks ports at the network edge)

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
# ⚠  Do NOT open 8004 or 3005 — nginx will proxy them internally.
sudo ufw --force enable
sudo ufw status
```

2. Install runtime dependencies

2.1 Python 3.11

Ubuntu 22.04 ships 3.10 — install 3.11:

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev
```

2.2 Node.js 20 LTS (needed by the Baileys sidecar)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo bash -
sudo apt install -y nodejs
node -v && npm -v
sudo npm install -g yarn
```

2.3 MongoDB 7 (Community Edition)

```
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc \
     | sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg --dearmor

echo "deb [signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg] https://repo.mongodb.org/apt/ubuntu $
     | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list

sudo apt update
sudo apt install -y mongodb-org
sudo systemctl enable --now mongod
sudo systemctl status mongod --no-pager | head -6
```

MongoDB is bound to `127.0.0.1:27017` by default (safe). Leave it there — the FastAPI backend will connect over localhost.

2.4 Nginx (reverse proxy + HTTPS)

```
sudo apt install -y nginx certbot python3-certbot-nginx
sudo systemctl enable --now nginx
```

2.5 Supervisor (keeps services alive)

```
sudo apt install -y supervisor
sudo systemctl enable --now supervisor
```

3. Get the application code onto the VPS

If you're using **Emergent's *Save to GitHub***, first push from the platform, then clone that repo onto the VPS:

```
sudo -iu wa_api          # switch to the dedicated user
cd /opt/wa_api
git clone https://github.com/<your-github-user>/<repo-name>.git .
```

If instead you have a **local ZIP export** of the codebase, upload it with `scp` and unzip:

```
# From your local machine
scp wa_api.zip wa_api@<VPS_IP>:/opt/wa_api/
# Then on the VPS as wa_api:
cd /opt/wa_api && unzip wa_api.zip && rm wa_api.zip
```

You should now have this structure:

```
/opt/wa_api/
├── backend/
├── frontend/
├── wa-sidecar/
└── ...
```

4. Configure environment variables

4.1 Backend .env

```
cd /opt/wa_api/backend
cp .env.example .env 2>/dev/null || true   # if provided
nano .env
```

Paste (**replace secrets with values from a password manager**):

```
# ── Database ──────────────────────────────────────
MONGO_URL="mongodb://127.0.0.1:27017"
DB_NAME="wa_api_db"

# ── Auth / JWT ────────────────────────────────────
JWT_SECRET="$(openssl rand -hex 48)"            # regenerate once, then keep
JWT_ACCESS_MINUTES=60
JWT_REFRESH_DAYS=14
ADMIN_EMAIL="admin@yourdomain.com"
ADMIN_PASSWORD="ChangeMeAfterFirstLogin!"

# ── Sidecar ───────────────────────────────────────
WA_SIDECAR_URL="http://127.0.0.1:3005"
SIDECAR_TOKEN="$(openssl rand -hex 32)"

# ── Public URLs (used in emails / QR renders) ──
FRONTEND_ORIGIN="https://wa.yourdomain.com"
```

⚠ **Generate the JWT_SECRET / SIDECAR_TOKEN with the `openssl` commands shown** — do NOT ship default values.

4.2 Sidecar .env

```
cd /opt/wa_api/wa-sidecar
nano .env
```

Paste (the token MUST match the backend's `SIDECAR_TOKEN`):

```
PORT=3005
SIDECAR_TOKEN="<paste same value as backend>"
BACKEND_INTERNAL_URL="http://127.0.0.1:8004"
```

4.3 Frontend .env (build-time only)

The frontend embeds the backend URL into the compiled JS bundle. Make sure it's the **public HTTPS URL**, not `localhost`:

```
cd /opt/wa_api/frontend
nano .env
```

```
REACT_APP_BACKEND_URL=https://wa.yourdomain.com
WDS_SOCKET_PORT=443
```

5. Install project dependencies

5.1 Backend (Python)

```
cd /opt/wa_api/backend
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip wheel
pip install -r requirements.txt
deactivate
```

5.2 Sidecar (Node)

```
cd /opt/wa_api/wa-sidecar
yarn install --production=false
```

5.3 Frontend build (produces static files served by Nginx)

```
cd /opt/wa_api/frontend
yarn install
yarn build
# Output: /opt/wa_api/frontend/build/
```

6. Configure Supervisor (process manager)

Return to a sudo-capable user:

```
exit    # leave wa_api shell if you're still in it
sudo nano /etc/supervisor/conf.d/wa_api_backend.conf
```

Paste:

```
[program:wa_api_backend]
command=/opt/wa_api/backend/.venv/bin/uvicorn server:app --host 127.0.0.1 --port 8004 --workers 2
directory=/opt/wa_api/backend
user=wa_api
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
stdout_logfile=/var/log/wa_api/backend.out.log
stderr_logfile=/var/log/wa_api/backend.err.log
environment=PATH="/opt/wa_api/backend/.venv/bin:/usr/bin:/bin"
```

Then:

```
sudo nano /etc/supervisor/conf.d/wa_api_sidecar.conf
```

```
[program:wa_api_sidecar]
command=/usr/bin/node index.js
directory=/opt/wa_api/wa-sidecar
user=wa_api
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
stdout_logfile=/var/log/wa_api/sidecar.out.log
stderr_logfile=/var/log/wa_api/sidecar.err.log
```

Create log dir + start services:

```
sudo mkdir -p /var/log/wa_api && sudo chown wa_api:wa_api /var/log/wa_api
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl status
# Both wa_api_backend and wa_api_sidecar should read RUNNING
```

Tail the logs while they boot to confirm no errors:

```
sudo tail -n 100 /var/log/wa_api/backend.err.log
sudo tail -n 100 /var/log/wa_api/sidecar.err.log
```

7. Configure Nginx (HTTPS + reverse proxy)

7.0 ⚠ Remove ANY previous nginx config that owns this domain

If you previously ran another installer (e.g. **VMP CRM installer**, cPanel, another Emergent project) on the same domain, its config **will keep serving the old page** and hide WA_API completely — because nginx picks the first `server_name` match. Clean it up before you continue:

```
# 1. See EVERY nginx config that mentions your target domain
sudo grep -rli "wa.yourdomain.com" /etc/nginx/

# 2. See what nginx is actually going to load (safe read-only)
sudo nginx -T | grep -B2 -A25 "server_name wa.yourdomain.com"

# 3. Disable the ones that are NOT this project. Example (replace filenames with what step 1 shows):
sudo rm -f /etc/nginx/sites-enabled/vmp-crm.conf
sudo rm -f /etc/nginx/sites-enabled/default          # only if it points to /var/www/html
# If the old server_name is inside a shared /etc/nginx/conf.d/*.conf file,
# open it and DELETE the entire `server { ... }` block for wa.yourdomain.com.

# 4. Also delete the leftover HTML root (or you'll get confused later):
sudo grep -rli "VMP CRM" /var/www /root /home 2>/dev/null
# Then move it out of the way:
sudo mv /var/www/vmp-installer /var/www/_vmp-installer.old   # rename, don't rm, until you're sure
```

7.1 Create the WA_API site file

Create the site file:

```
sudo nano /etc/nginx/sites-available/wa_api.conf
```

Paste:

```
server {
    listen 80;
    server_name wa.yourdomain.com;

    # Serve the compiled React SPA
    root /opt/wa_api/frontend/build;
    index index.html;

    # Everything /api goes to FastAPI
    location /api/ {
        proxy_pass         http://127.0.0.1:8004;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_set_header   Upgrade           $http_upgrade;
        proxy_set_header   Connection        "upgrade";
        proxy_read_timeout 3600;
        client_max_body_size 60m;      # allow media uploads up to ~50 MB
    }

    # SPA fallback
    location / {
        try_files $uri /index.html;
    }

    # Long cache for static assets
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff2?)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

Enable, test, reload:

```
sudo ln -s /etc/nginx/sites-available/wa_api.conf /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

7.2 ⚠ Verify nginx is really serving WA_API (not the old site)

Before you enable HTTPS, run these three commands. All three MUST pass — otherwise HTTPS will get issued for the wrong site and you'll be stuck.

```
# a) Confirm the config that owns the domain points to /opt/wa_api/frontend/build
sudo nginx -T | grep -B1 -A20 "server_name wa.yourdomain.com" \
     | grep -E "root|proxy_pass"
# Expected output must include:
#   root /opt/wa_api/frontend/build;
#   proxy_pass http://127.0.0.1:8004;

# b) Hit the site over plain HTTP — should serve the WA_API landing page HTML (contains "WA_API")
curl -s http://wa.yourdomain.com/ | grep -o "WA_API\|VMP CRM" | head -1
# Expected: WA_API      ([X] if you see "VMP CRM", step 7.0 wasn't done — go back!)

# c) Hit /api/health — MUST return JSON, not HTML
curl -s http://wa.yourdomain.com/api/health
# Expected: {"ok":true,"sidecar":true,"version":"2.0.0"}
# [X] If you see "<!DOCTYPE html>" instead, nginx is NOT proxying /api → 8004.
#    That means (i) the wa_api config isn't the active one, or (ii) the
#    `location /api/` block is missing / wrong. Fix and re-run step 7.
```

Point your DNS `A` record `wa.yourdomain.com → <VPS_IP>` and wait for propagation.

7.1 Enable HTTPS with Let's Encrypt

```
sudo certbot --nginx -d wa.yourdomain.com --agree-tos -m admin@yourdomain.com --redirect --non-interac
sudo systemctl enable --now certbot.timer     # auto-renew twice a day
```

8. First-run checks

8.1 Health check

```
curl -sI https://wa.yourdomain.com/api/health
# HTTP/2 200
```

8.2 Marketing site loads

Open `https://wa.yourdomain.com` in a browser — you should see the WA_API landing page (mobile-responsive).

8.3 Admin login

Go to `https://wa.yourdomain.com/login` and sign in with the `ADMIN_EMAIL` / `ADMIN_PASSWORD` you set in `backend/.env`. The seed script runs on first backend start and creates that admin.

Change the admin password immediately — sidebar → **Password** button.

8.4 Configure SMTP (optional but recommended)

`/admin/smtp` → paste your Gmail app-password or transactional SMTP creds → **Send test**. Once this works, welcome emails and password-reset OTPs will be delivered.

8.5 Create a WhatsApp session

`/app/sessions` → **NEW SESSION** → scan the QR from your phone (**WhatsApp → Linked devices**). Within seconds the badge flips to `CONNECTED`.

8.6 Verify the public API

Create an API key on `/app/keys`, then from another machine:

```
curl -H "Authorization: Bearer <YOUR_KEY>" \
     https://wa.yourdomain.com/api/v1/sessions/primary/status
# {"id":"primary","connected":true,"status":"connected","phone":"...","checked_at":"..."}
```

9. Operational cheat-sheet

Task	Command
Tail backend logs	sudo tail -f /var/log/wa_api/backend.err.log
Tail sidecar logs	sudo tail -f /var/log/wa_api/sidecar.err.log
Restart backend	sudo supervisorctl restart wa_api_backend
Restart sidecar	sudo supervisorctl restart wa_api_sidecar
Restart Nginx	sudo systemctl reload nginx
Rebuild frontend after code change	cd /opt/wa_api/frontend && yarn build && sudo systemctl relo
Update code (pull latest & restart)	cd /opt/wa_api && git pull && cd backend && source .venv/bir
MongoDB shell	mongosh wa_api_db
Backup Mongo	mongodump --db wa_api_db --out /var/backups/mongo_$(date

10. Ports summary (multi-tenant VPS)

Project	Backend port	Sidecar / other

Emergent project #1	8001	—
Emergent project #2	8002	—
Emergent project #3	8003	—
WA_API (this)	8004	3005 (Node/Baileys)

All internal ports are bound to `127.0.0.1` — the only public ports remain **80 / 443** exposed by Nginx. Each project keeps its own DB (`wa_api_db`) on the shared local Mongo instance.

11. Security hardening (production)

1. **Disable password SSH** — after adding your public key, edit `/etc/ssh/sshd_config` → `PasswordAuthentication no` → `sudo systemctl restart ssh`.
2. **Fail2ban** is already installed; default rules cover SSH out-of-the-box.
3. **MongoDB auth** — for shared Mongo, enable auth: create an admin user in mongosh, then set `security: authorization: enabled` in `/etc/mongod.conf` and update the connection string to `mongodb://user:pass@127.0.0.1:27017/wa_api_db`.
4. **Regularly update packages** — `sudo unattended-upgrades` handles kernel & OpenSSL patches.
5. **Rotate SIDECAR_TOKEN & JWT_SECRET** every 90 days — restart backend + sidecar together.
6. **Never expose 8004 / 3005** to the public — always keep Nginx in front.

12. Troubleshooting

Backend won't start — `sudo tail -n 200 /var/log/wa_api/backend.err.log`. Typical fixes:

- Wrong Python version → use `python3.11`, recreate `.venv`.
- Mongo not running → `sudo systemctl status mongod`.

Sidecar won't start — `sudo tail -n 200 /var/log/wa_api/sidecar.err.log`. Typical fixes:

- Port 3005 in use → `sudo ss -tlnp | grep 3005` and stop the offender or pick another port.
- `SIDECAR_TOKEN` mismatched — must be identical in both `.env` files.

QR code never renders — the sidecar can't reach WhatsApp servers. Check DNS from the VPS (`curl -I https://web.whatsapp.com`). Some cheap VPS providers block outbound WebSocket traffic — contact support if so.

"Not connected" showing in the CRM even after scanning — hit `GET /api/v1/sessions/<slug>/status`. If `sidecar_reachable=false` your backend can't reach the

sidecar → check supervisor status and the `WA_SIDECAR_URL` env value.

502 from Nginx - 9 times out of 10 the backend crashed. `supervisorctl status wa_api_backend`, tail the log, fix, restart.

Domain shows a DIFFERENT project (e.g. old VMP CRM installer) - your VPS still has an older nginx server-block claiming the same `server_name`. Nginx returns the first match, so WA_API is invisible.

Diagnose:

```
sudo grep -rli "wa.yourdomain.com" /etc/nginx/            # every config that owns the domain
sudo nginx -T | grep -B1 -A25 "server_name wa.yourdomain.com" | grep -E "root|proxy_pass"
```

You should see exactly ONE `root /opt/wa_api/frontend/build;` and one `proxy_pass http://127.0.0.1:8004;`. If you see a `root /var/www/vmp-installer;` (or any other path) or a competing server block, disable it and reload:

```
sudo rm -f /etc/nginx/sites-enabled/vmp-crm.conf              # or the offending file
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx

# Re-verify:
curl -s http://wa.yourdomain.com/api/health       # must return JSON
curl -s http://wa.yourdomain.com/ | head -c 400   # must contain "WA_API"
```

If `curl https://wa.yourdomain.com/api/health` returns HTML but the plain-HTTP variant works, `certbot` installed the certificate for the WRONG site earlier. Re-run:

```
sudo certbot delete --cert-name wa.yourdomain.com
sudo certbot --nginx -d wa.yourdomain.com --agree-tos -m admin@yourdomain.com --redirect --non-interac
```

13. Upgrading the app

Every code change should follow this order:

```
sudo -iu wa_api
cd /opt/wa_api
git pull

# Python deps
cd backend && source .venv/bin/activate
pip install -r requirements.txt
deactivate

# Node deps
cd ../wa-sidecar && yarn install --production=false
cd ../frontend && yarn install

# Build UI
yarn build

# Back to root user to restart
exit
sudo supervisorctl restart wa_api_backend wa_api_sidecar
sudo systemctl reload nginx
```

Bookmark this section — every future release update is exactly the same recipe.

End of guide. Keep this PDF and your `.env` files in a password manager. If you rebuild the VPS from scratch, sections **1 – 8** get you back online in under 30 minutes.